

Remote Assistance Setup Guide — Server (Host) Machine

This guide covers setting up the **target/host machine** (the Xubuntu computer that will receive remote assistance) to allow secure, on-demand screen sharing over Tailscale, using a single shared password that works regardless of which user is logged in.

Prerequisites

- Xubuntu machine with Tailscale already installed and connected
- SSH server installed
- x11vnc installed

Install what's missing:

```
sudo apt update
sudo apt install openssh-server x11vnc
```

Step 1: Create the Shared VNC Password

This creates one password that will work for connecting to any user's session on this machine.

```
sudo x11vnc -storepasswd /etc/x11vnc.pass
sudo chmod 600 /etc/x11vnc.pass
sudo chown root:root /etc/x11vnc.pass
```

You will be prompted to set a password. This is the password anyone connecting will need to enter, regardless of which user is currently logged into the machine.

Step 2: Create the Launch Script

This script automatically detects the currently active user and X display, then starts x11vnc attached to that session.

```
sudo nano /usr/local/bin/start-remote-assist.sh
```

Paste in the following:

```
#!/bin/bash

if [ "$EUID" -ne 0 ]; then
    echo "Please run with sudo"
    exit 1
fi

# Extract user and display from `who` output
WHO_LINE=$(who | grep -m1 '(:[0-9]\+)' )
```

```

if [ -z "$WHO_LINE" ]; then
    echo "No active X session found, exiting"
    exit 1
fi

ACTIVE_USER=$(echo "$WHO_LINE" | awk '{print $1}')
ACTIVE_DISPLAY=$(echo "$WHO_LINE" | grep -oP '(?<=\\()[^)]+(?=\\)')

if [ -z "$ACTIVE_USER" ] || [ -z "$ACTIVE_DISPLAY" ]; then
    echo "Could not parse user/display, exiting"
    exit 1
fi

echo "Detected user: $ACTIVE_USER, display: $ACTIVE_DISPLAY"

XAUTH_PATH="/home/$ACTIVE_USER/.Xauthority"

if [ ! -f "$XAUTH_PATH" ]; then
    echo "Xauthority not found at $XAUTH_PATH, exiting"
    exit 1
fi

# Start x11vnc in the background so the script can keep running
x11vnc \
    -display "$ACTIVE_DISPLAY" \
    -auth "$XAUTH_PATH" \
    -rfbauth /etc/x11vnc.pass \
    -localhost \
    -forever \
    -noxdamage &

VNC_PID=$!

echo ""
echo "Remote assistance session is active on localhost:5900."

# Prompt loop - keeps asking until 'y' is entered
while true; do
    read -rp "Press 'y' then Enter to end the connection: " input
    if [[ "$input" == "y" || "$input" == "Y" ]]; then
        kill "$VNC_PID" 2>/dev/null
        wait "$VNC_PID" 2>/dev/null
        echo "Connection closed."
        break
    fi
done

```

Save the file, then make it executable:

```
sudo chmod 700 /usr/local/bin/start-remote-assist.sh
```

What each part does

Flag / Section	Purpose
who parsing	Automatically finds whichever user is logged in on display :0, so it works no matter who's signed in

<code>-auth "\$XAUTH_PATH"</code>	Explicitly points to that user's .Xauthority file (more reliable than <code>-auth guess</code>)
<code>-rfbauth /etc/x11vnc.pass</code>	Uses the one shared password file from Step 1
<code>-localhost</code>	Only accepts connections through a tunnel (SSH/Tailscale) — never exposed directly to the network
<code>-forever</code>	Keeps listening for new connections instead of exiting after one session
<code>-noxdamage</code>	Avoids a known issue with the X DAMAGE extension when attaching across sessions
Prompt loop	Keeps the session open until you manually type <code>y</code> and press Enter, giving you full control over when the session ends

Step 3: Run the Script Whenever You Want to Allow a Remote Assistance Session

```
sudo /usr/local/bin/start-remote-assist.sh
```

Expected output:

```
Detected user: yourusername, display: :0
```

```
Remote assistance session is active on localhost:5900.  
Press 'y' then Enter to end the connection:
```

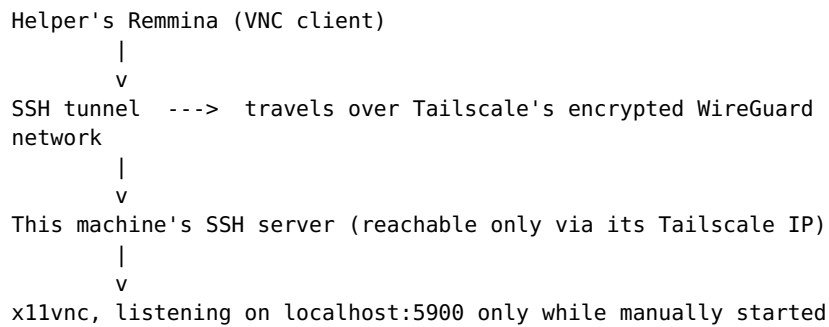
Leave this terminal open for the duration of the session. The helper can now connect (see the client-side/Remmina setup for that half). When you're done, type `y` and press Enter in this terminal to close the connection.

Step 4 (Recommended): Restrict Access via Tailscale ACLs

By default, a new Tailscale network allows all devices in your tailnet to reach each other. To limit exposure, edit your tailnet policy file in the Tailscale admin console so that only the specific helper account/device can reach this machine, ideally restricted to port 22 (SSH) only.

This ensures that even if another device joins your tailnet in the future, it cannot reach this machine's SSH port or, by extension, the VNC session behind it.

How the Full Chain Fits Together



- **Tailscale** provides the private, encrypted network connecting the two machines — no ports are opened on your router, and nothing is exposed to the public internet.
- **SSH** provides an authenticated, encrypted tunnel through which the VNC traffic travels.
- **x11vnc** shares the actual live desktop session, gated by `-localhost` (only reachable through the tunnel) and the shared password.

Quick Reference Summary

Component	Behavior
Password	One shared file (/etc/x11vnc.pass) — works for any logged-in user
Session targeting	Automatically detects and attaches to the currently active user's display
Network exposure	None — <code>-localhost</code> blocks all direct connections
Session control	Manually started and stopped via the terminal script (press <code>y</code> to end)
Access restriction	Enforced via Tailscale ACLs, limiting which devices can reach this machine at all